# 1. Introduction of Project

This project focuses on the **automation of deployment** for an **HRMS (Human Resource Management System)** application using modern **DevOps practices**. The primary objective is to manage the complete lifecycle of the application — from building and testing to deployment — in a structured, efficient, and repeatable manner.

Traditionally, application deployment is performed manually, which can lead to errors, inconsistencies, and increased deployment time. To overcome these challenges, this project implements an **automated deployment pipeline** that ensures application updates are delivered smoothly, reliably, and consistently across the target environment.

The system integrates various DevOps tools and methodologies to streamline the software delivery process. By automating tasks such as code integration, build generation, testing, and deployment, the project minimizes human intervention and reduces the risk of configuration or deployment errors.

Furthermore, this automated approach improves collaboration between development and operations teams, enhances deployment speed, and enables faster delivery of new features and bug fixes. Overall, the project demonstrates how DevOps practices can be effectively applied to maintain consistency, improve reliability, and ensure efficient management of application deployment for an HRMS system.

## 2. Current / Existing System

In the existing system, the deployment and management of web applications are performed manually. Each component of the application, including the front-end, back-end, and database, is deployed and managed separately on different servers or virtual machines, requiring individual configuration and maintenance.

In traditional web application environments, applications are typically deployed on a single server or virtual machine where most deployment and configuration tasks are handled manually. Activities such as installing dependencies, configuring the environment, updating application code, and managing services depend largely on manual effort.

This manual approach increases the time required for deployment and makes the process less efficient. It also creates challenges in maintaining consistency across different environments, as each deployment may follow slightly different steps. As a result, the existing system is more prone to errors, difficult to manage, and less reliable for frequent application updates.



### a) Flow/Process

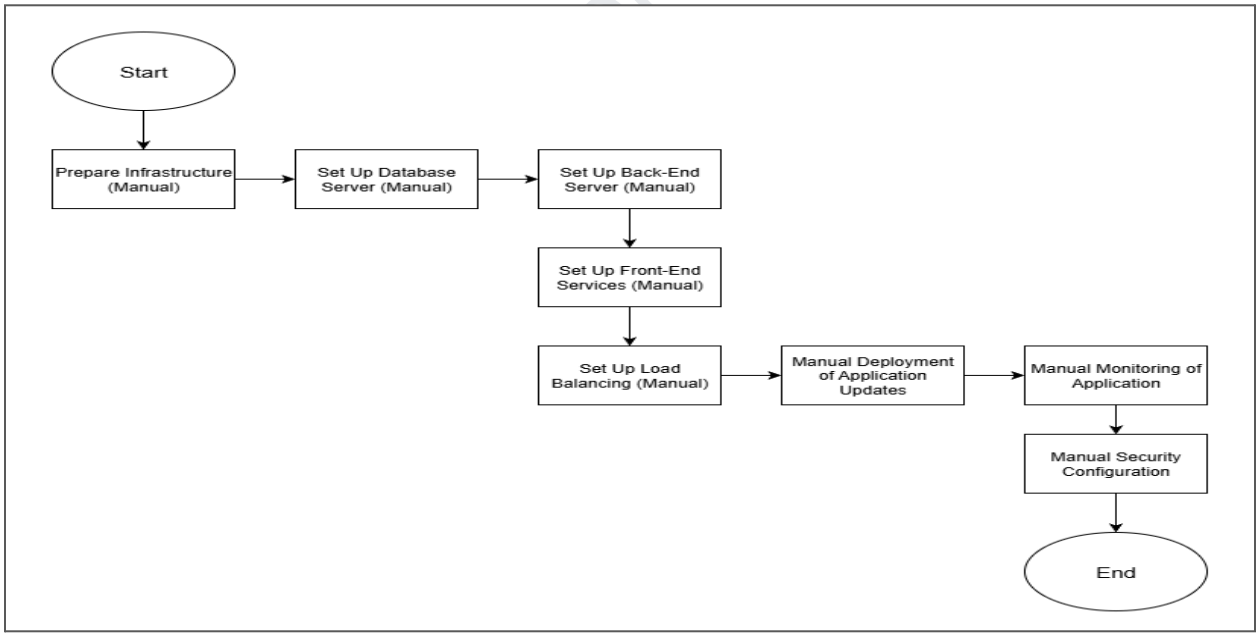



fig. 1: Existing System Flow

**b) Drawbacks**

**(i) Time-Consuming Deployment**

Manual setup and deployment require significant time because each step — including configuration, installation, and deployment — must be performed manually by an individual. This slows down the overall release process.

**(ii) High Chance of Human Errors**

Since configuration and deployment activities are handled manually, there is a higher risk of mistakes such as incorrect settings, missed steps, or version mismatches, which may lead to application failures.

**(iii) Difficult to Manage Multiple Components**

Managing different components of the application, such as the front-end, back-end, and database, separately becomes complex without automation. Coordinating updates and configurations across these components is challenging.

**(iv) Limited Scalability**

The system cannot easily scale to handle increased users or workload because scaling requires manual server provisioning and configuration. This makes it difficult to respond quickly to growing business needs.

**(v) Lack of Consistency**

Manual setup often results in differences between development, testing, and production environments. These inconsistencies may cause unexpected behavior when the application is deployed.

**(vi) Slow Application Updates**

Application updates take more time because deployment depends on manual intervention. Repeating the same steps for every update delays feature releases and bug fixes.

**(vii) No Continuous Monitoring**

Without automated monitoring, system issues or performance problems may not be detected immediately. This can increase downtime and impact application reliability.

**(viii) Higher Maintenance Effort**

Maintaining servers, applications, and security settings requires continuous manual work. This increases operational effort and makes long-term system maintenance more difficult.

## 3. Proposed System

The proposed system introduces automation in the deployment and management of the **HRMS (Human Resource Management System)** web application by implementing DevOps practices. Instead of relying on manual processes, the system uses an automated workflow to handle key stages of the application lifecycle.

In this approach, important tasks such as building, testing, and deploying the application are executed automatically through a well-defined pipeline. Whenever changes are made to the application source code, the system automatically detects those changes and processes them without requiring manual intervention. This ensures faster and more efficient delivery of application updates.

Automation plays a crucial role in maintaining consistency across different environments, including development, testing, and production. The proposed system ensures that the application is deployed in a reliable, repeatable, and standardized manner. It also reduces configuration-related issues that commonly occur in manual deployment.

By minimizing human involvement, the proposed system saves time, improves operational efficiency, and reduces the chances of errors. As a result, the overall reliability of the system increases, and the deployment process becomes faster, smoother, and easier to manage. This approach supports continuous delivery and enables organizations to release updates more frequently with greater confidence.

**a) Modules:**

**(i) Source Code Management Module**
This module manages the application source code using GitHub. It provides version control by tracking code changes, maintaining version history, and supporting collaboration among team members. Developers can push, pull, and manage code efficiently, ensuring proper control over the development process.

**(ii) Continuous Integration Module**
The Continuous Integration (CI) module automatically fetches the latest source code and performs build and validation processes using Jenkins. It ensures that every code change is integrated, tested, and verified before moving to the next stage, helping detect issues early in the development cycle.

**(iii) Code Quality Module**
This module focuses on analyzing the quality of the application code using SonarQube. It helps identify bugs, security vulnerabilities, and code smells while ensuring that coding standards and best practices are followed. This improves maintainability and reliability of the application.

**(iv) Artifact Management Module**

The Artifact Management module creates build packages after successful integration and stores them in the Nexus repository. It enables versioned storage of artifacts, making it easy to reuse, track, and manage different application versions during deployment.

**(v) Deployment and Infrastructure Module**

This module automates the deployment of the application to AWS EC2 using Jenkins. It manages the server environment by configuring infrastructure components such as EC2 instances and Nginx for application hosting. Automation ensures faster, consistent, and reliable deployment across environments.

**b) Features:**

- **Automated Deployment**
  The application is automatically built, tested, and deployed through a defined DevOps pipeline without requiring manual intervention. This ensures a smooth and standardized deployment process.
- **Reduced Manual Effort**
  Automation minimizes the need for human involvement in repetitive tasks such as configuration, build execution, and deployment. This saves time and reduces operational workload.
- **Faster Application Updates**
  Application changes can be deployed quickly whenever updates are made to the source code. The automated workflow enables faster delivery of new features and bug fixes.
- **Improved Reliability**
  Automated processes reduce the chances of human errors and ensure that each deployment follows the same steps. This increases system reliability and stability.
- **Consistent Application Environment**
  The system maintains consistency across development, testing, and production environments. This prevents configuration-related issues and ensures predictable application behavior.
- **Easy Monitoring**
  The proposed system supports easy monitoring of application status, performance, and deployment activities. This helps in quickly identifying and resolving issues.
- **Better Resource Utilization**
  Automation helps in efficient use of system resources such as servers, storage, and memory. It optimizes infrastructure usage and reduces unnecessary resource consumption.

- **Scalable System**
  The system is designed to handle increased user load and future expansion. With automated infrastructure and deployment processes, scaling can be achieved with minimal changes and effort.

**c) Scope:**

The scope of this project is to implement an automated and reliable DevOps pipeline for deploying and managing the HRMS application efficiently. The system focuses on reducing manual effort by automating the build, testing, and deployment processes, ensuring that application updates can be delivered quickly and consistently.

The project supports multi-component applications by allowing different parts of the application to be integrated and deployed in an organized manner. It provides a structured workflow that helps in managing application updates, configurations, and deployment activities in a controlled and efficient way.

The system improves deployment speed by minimizing manual steps and enabling faster release of application changes. This results in better system reliability, as automated processes reduce configuration errors and ensure consistent deployments across environments.

The scope also includes cloud-based deployment support, where the application can be hosted and managed on cloud infrastructure such as AWS. This allows the system to be flexible and accessible from different environments.

In addition, the project enables continuous monitoring of the application and server performance, helping maintain system stability and availability. The solution is designed to be scalable, allowing future expansion as application usage grows or additional services are integrated.

Overall, the scope of the project is to create a modern DevOps-based deployment environment that improves application delivery, reliability, scalability, and operational efficiency.

## ❖ Automated CI/CD Workflow using DevOps

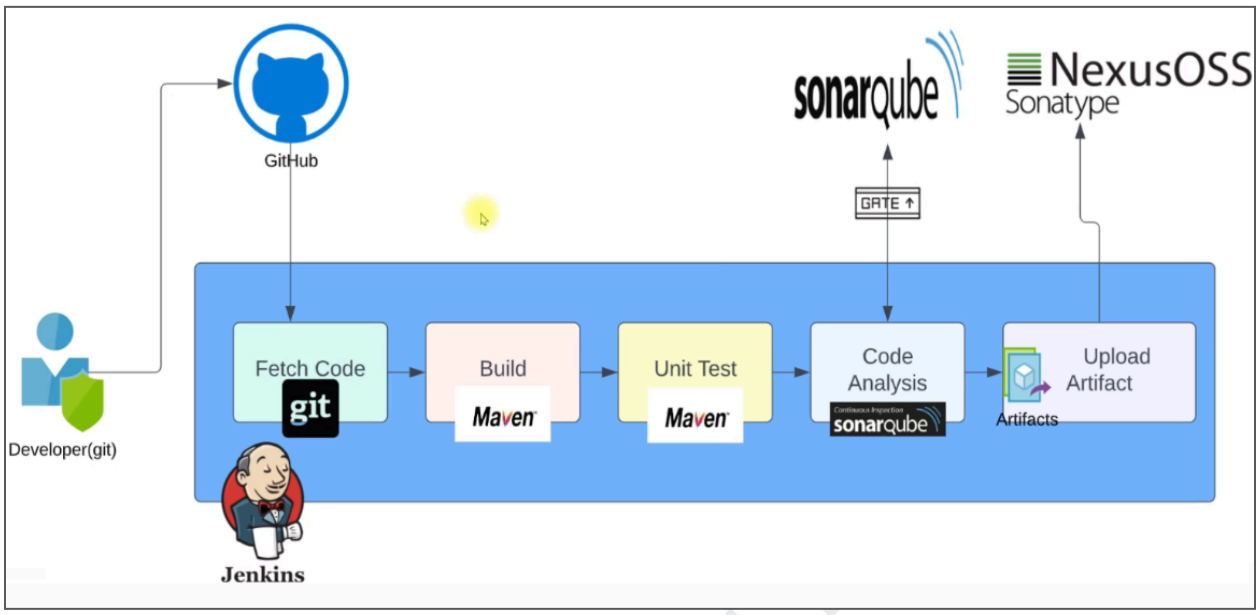



fig 2: workflow

Suppose there is an HRMS web application already running for a company.
Now, the client requests a small change, such as adding a new feature or modifying an existing page.

- **Developer Writes Code:** The developer makes the required changes in the application code on their local system. After completing the changes, the developer pushes the updated code to GitHub, which is used as a central code storage system.

- **Fetch Code Automatically:** Once the code is pushed to GitHub, the DevOps system automatically fetches the latest code from the repository. This avoids the need for manually copying code to the server.

- **Build the Application:** After fetching the code, the system builds the application automatically.
  This step prepares the application so that it can be executed properly.

- **Unit Testing:** Next, basic tests are executed automatically to check whether the new changes have caused any issues. If tests fail, the process stops and the issue is reported.

- **Code Analysis:** The code is then checked for quality and standard practices. This helps in identifying bugs or poor coding practices before deployment.

- **Upload Artifact:** After successful build and testing, the final application file (called an artifact) is generated. This artifact is stored securely so that it can be deployed whenever required.

**Role of DevOps Engineer:**

- Setting up this automated process
- Monitoring the build and deployment steps
- Ensuring smooth and error-free application updates

## 4. Hardware and Software Specification

### Hardware Specification

| Hardware Component | Description |
|---|---|
| System / Laptop | Used for development and configuration |
| Processor | Intel i5 or equivalent |
| RAM | Minimum 8 GB |
| Storage | Minimum 256 GB |
| Server (Local / Cloud) | Used for application deployment |

### Software Specification

| Software Component | Description |
|---|---|
| Operating System | Linux / Windows |
| Automation Tool | Jenkins |
| Containerization Tool | Docker |
| IDE | Visual Studio Code (VS Code) |
| Web Browser | Google Chrome/Safari/Edge/Etc |
| Monitoring Tools | Cloudwatch, Prometheus, Grafana |
| Version Control Tool | GitHub |
| Cloud Computing Platform | Amazon Web Services (AWS) |

## ❖ Tools and Technologies Used

**(i) Jenkins – Automation Tool**

Jenkins is an open-source automation tool used to build and manage Continuous Integration and Continuous Deployment (CI/CD) pipelines. It automatically executes tasks whenever code changes are detected in the repository. Jenkins integrates with multiple tools such as GitHub, SonarQube, Nexus, and AWS, enabling a complete DevOps workflow. With its wide range of plugins, Jenkins offers flexibility and acts as the central controller of the entire automation pipeline.

**(ii) SonarQube – Code Quality Analysis**

SonarQube is a code quality analysis tool that evaluates the application source code to maintain high coding standards. It detects bugs, security vulnerabilities, and duplicate code while identifying code smells. SonarQube uses Quality Gates to determine whether the code meets defined quality criteria, ensuring that only clean, secure, and reliable code proceeds to deployment.

**(iii) Nexus Repository – Artifact Storage**

Nexus Repository is an artifact repository manager used to store build outputs such as JAR files, WAR files, and Docker images. It maintains version history of build artifacts and serves as a central storage location for deployment packages. This makes it easier to manage different versions, reuse artifacts, and perform rollback when required.

**(iv) Prometheus – Monitoring and Metrics Storage**

Prometheus is a monitoring and alerting toolkit that collects and stores time-series metrics from applications and infrastructure components. It gathers data by scraping metrics from exporters and service endpoints. Prometheus allows alerting based on predefined rules and provides powerful querying capabilities through PromQL for performance analysis and system monitoring.

**(v) Grafana – Visualization and Dashboards**

Grafana is a visualization and analytics platform used to display metrics collected by Prometheus and other data sources. It enables the creation of interactive dashboards for monitoring system performance and application health. Grafana supports alerts, annotations, and reporting features, providing a centralized view of operational data and helping teams analyze trends and troubleshoot issues effectively.

# 5. Analysis

## a) Fact Finding Methods

**Discussion with Development and Operations Team**

Discussions were conducted with the development and operations teams to understand the existing application deployment process and the challenges faced during manual deployment. These interactions helped in identifying key deployment requirements and areas where automation could improve efficiency and reliability.

**Study of Existing System Architecture**

The current system architecture was analyzed to understand how different components of the HRMS application are deployed and managed. This included reviewing server configurations, application structure, and the overall deployment workflow used in the existing environment.

**Observation of Existing Deployment Process**

The manual deployment process was carefully observed to analyze how application updates are performed and maintained. This observation helped identify major limitations such as delays, configuration errors, repetitive tasks, and increased operational complexity.

**Prototyping of Automated Deployment**

A small prototype of the automated deployment process was developed to evaluate how automation can improve deployment efficiency. The prototype allowed testing of the CI/CD workflow in a controlled environment before implementing it in the full system.

**Use Case Analysis**

Use case analysis was performed to understand how different users and system components interact with the HRMS application. This ensured that the proposed automated deployment system supports real operational scenarios and meets user requirements.

**Resource Requirement Analysis**

The required hardware and software resources — including CPU, memory, storage, and DevOps tools — were identified. This analysis helped in planning the infrastructure needed to support the automated deployment system effectively.

## b) Process Model, Development Strategy, Type of System

- **Process Model: DevOps Model**

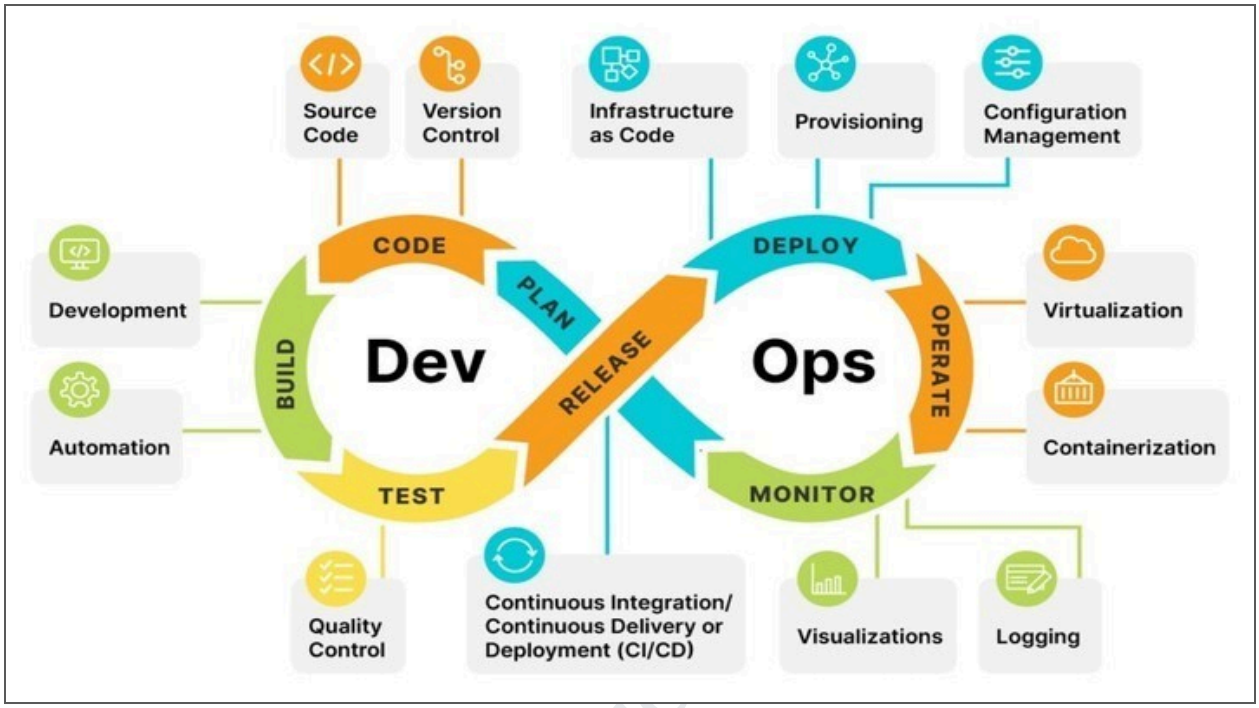



fig 3: Process Model

This diagram represents the **DevOps lifecycle** using an **infinity loop**, showing the continuous development, deployment, and operation of software. The left side of the loop represents **Development (Dev)**, and the right side represents **Operations (Ops)**.

**1. Plan:** In this step, the team plans the work to be done. Requirements, tasks, and timelines are discussed before development starts.

**2. Code:** Developers write and update the application source code. The code is stored and managed using version control systems.

**3. Build:** The written code is converted into a runnable application. Any required files or dependencies are added in this step.

**4. Test:** The application is tested to check for errors or issues. This helps ensure the application works correctly before deployment.

**5. Release:** The tested application is prepared for deployment. This step ensures the correct version is ready to be released.

**6. Deploy:** The application is deployed on the server or environment. Deployment is done automatically to reduce manual effort.

**7. Operate:** The deployed application is kept running smoothly. Daily operations like server management and updates are handled here.

**8. Monitor:** The application is monitored to check performance and errors. This helps in identifying issues early and improving reliability.

- ## Development Strategy

The deployment strategy of this project focuses on automating the deployment of the HRMS web application using DevOps practices. The application is deployed in a structured and organized manner, where different parts of the system are handled separately to improve maintainability, flexibility, and reliability.

**Component-Based Deployment**

The application is divided into logical components such as front-end, back-end, and database. Each component is deployed and managed independently, making it easier to update specific parts of the system without affecting the entire application. This approach also supports better maintenance and scalability.

**Automated Build and Deployment**

Whenever changes are made to the application source code, the automated pipeline triggers the build and deployment process. The system automatically builds the updated version, performs necessary validations, and deploys it to the target environment. This reduces manual effort and ensures faster, more reliable, and error-free deployments.

**Consistent Deployment Environment**

The deployment process ensures that the application runs in a consistent environment across development, testing, and production stages. Standardized configuration and automated setup help prevent environment-related issues and improve deployment predictability.

**Monitoring and Maintenance**

Basic monitoring mechanisms are implemented to observe application status, deployment results, and system performance. Continuous monitoring helps maintain system stability, quickly detect issues, and support efficient maintenance of the deployed application.

- **Type of System**

The proposed system is a **Web-Based Human Resource Management System (HRMS)** that provides various employee-related services through a centralized platform. It enables employees to access and manage their HR information online, reducing dependency on manual processes and improving accessibility.

Through this system, employees can view important HR details such as the company holiday list, check the number of available leaves in their account, and apply for leave directly through the web interface. This simplifies leave management and ensures that requests are processed in a structured manner.

The system also allows employees to view attendance information, including daily working hours and work records. This helps employees track their attendance and maintain transparency in their work history. HR teams can also manage and monitor employee data more efficiently using the same platform.

All services are provided through a web-based interface, making the system easy to use, accessible from different locations, and user-friendly. Overall, the HRMS improves communication between employees and HR departments while streamlining routine HR operations.

## c) Diagrams:

### 1. Estimated Gantt Chart/Time Line Chart:

| Tasks | Week 1 | Week 2 | Week 3 | Week 4 | Week 5 | Week 6 | Week 7 | Week 8 | Week 9 | Week 10 | Week 11 | Week 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | | | | |
| Discuss with Development Team | | | | | | | | | | | | |
| Developed HRMS for Demo | | | | | | | | | | | | |
| Set up Jenkins Server | | | | | | | | | | | | |
| Set up SonarQube and Nexus Server | | | | | | | | | | | | |
| Prometheus and Grafana | | | | | | | | | | | | |
| Deployed app Locally | | | | | | | | | | | | |
| Deployed app on Server and Check | | | | | | | | | | | | |

### 2. UML Diagrams:

### (A)Use Case Diagram

(i) Developer

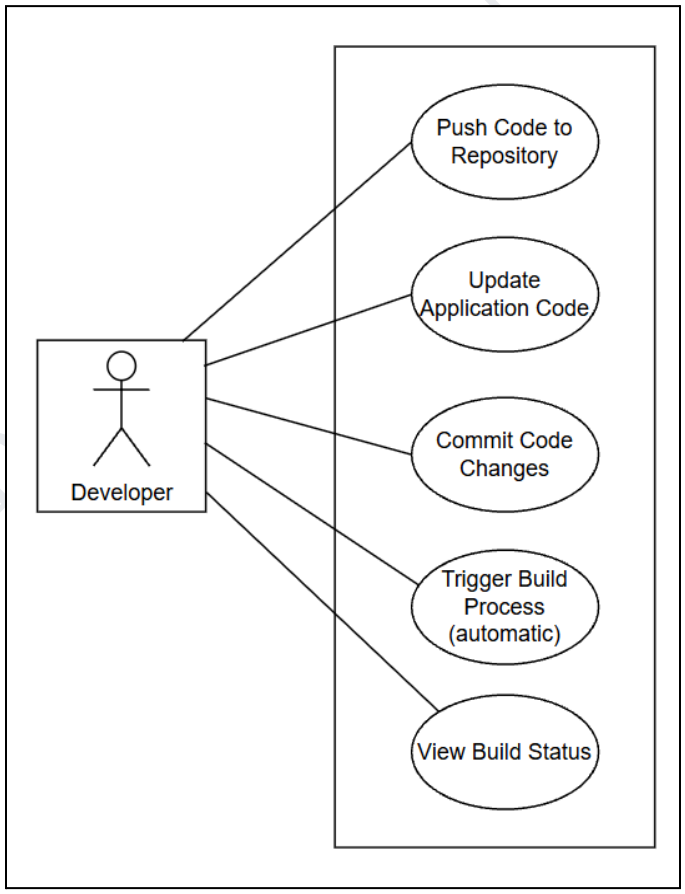



fig 4: Use Case diagram (Developer)

(ii) DevOps Engineer

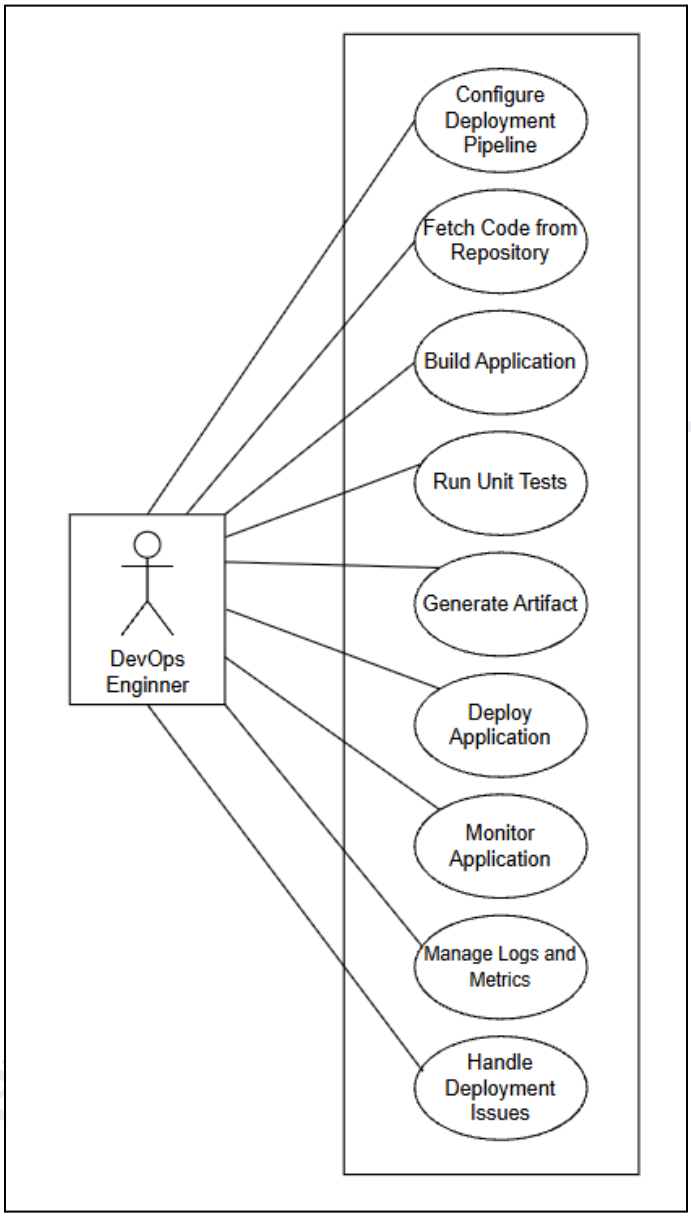



fig 5: Use Case diagram (DevOps)

(iii) User

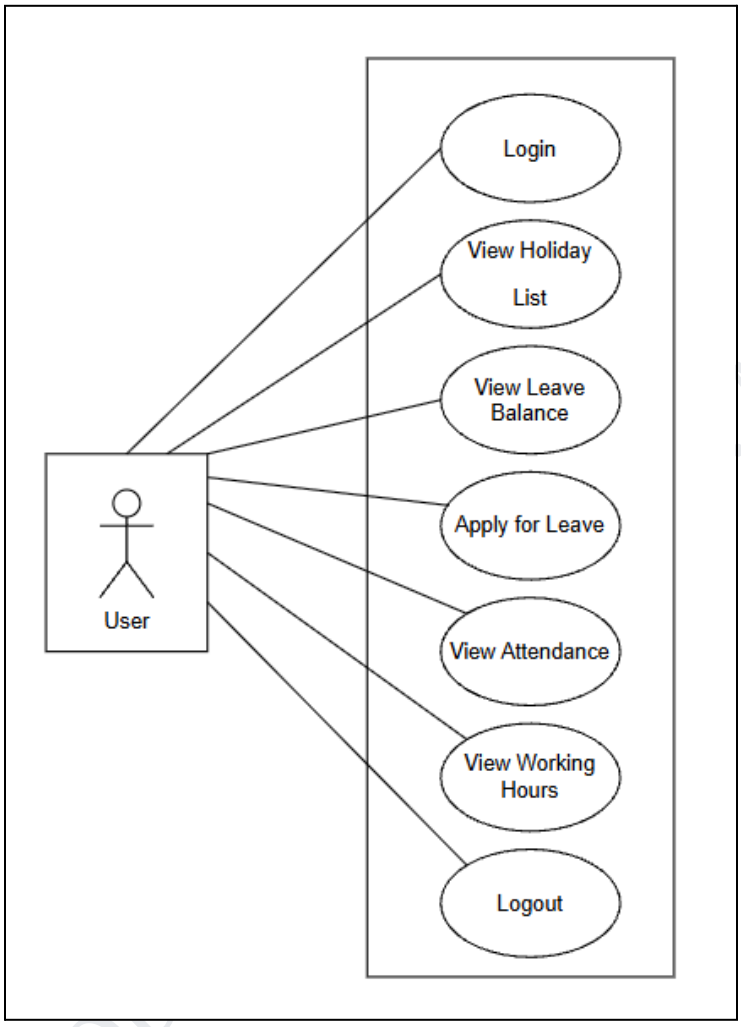



fig 6: Use Case diagram (User)

This is a Use Case Diagram that represents the interactions between different users and the HRMS application deployment system. The diagram shows how various users interact with the system during application deployment, management, and usage.

**Actors:**

**Developer** :
A user responsible for writing, updating, and committing application code, and pushing the code to the repository to initiate the automated build and deployment process.

**DevOps Engineer** :
A technical user responsible for configuring and managing the deployment pipeline, building the application, deploying it to the server, monitoring system performance, and managing logs and metrics of the HRMS application.

**End User** :

A general user (employee) who accesses the deployed HRMS application to log in, view HR-related information such as holidays, leave balance, attendance, working hours, and log out of the system.

**(B) Activity Diagram**

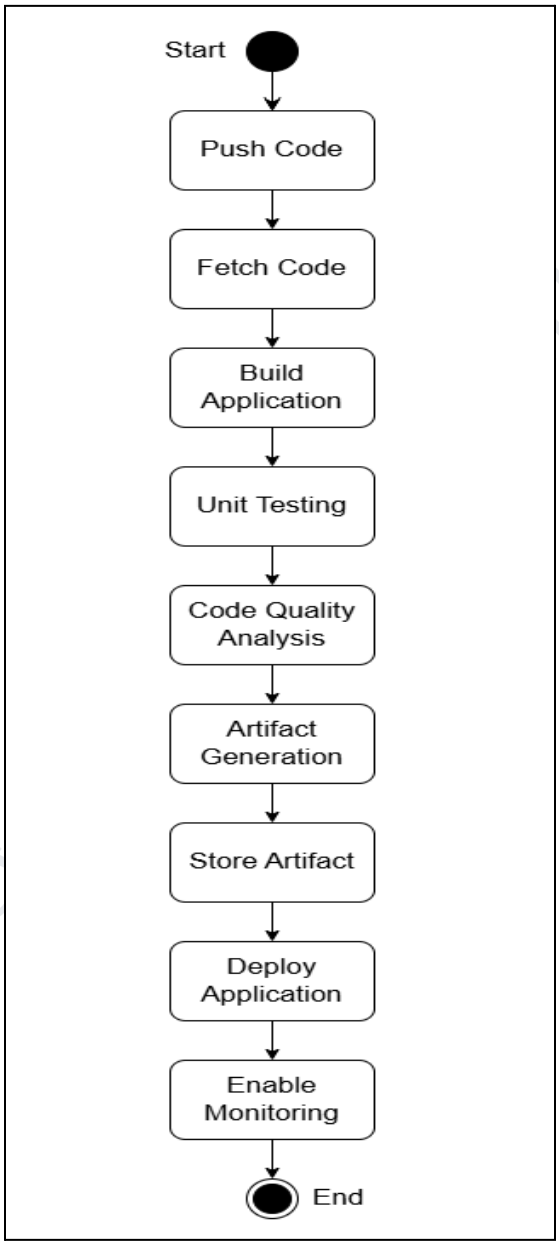



fig 7: Activity diagram

**(c) Class Diagram**

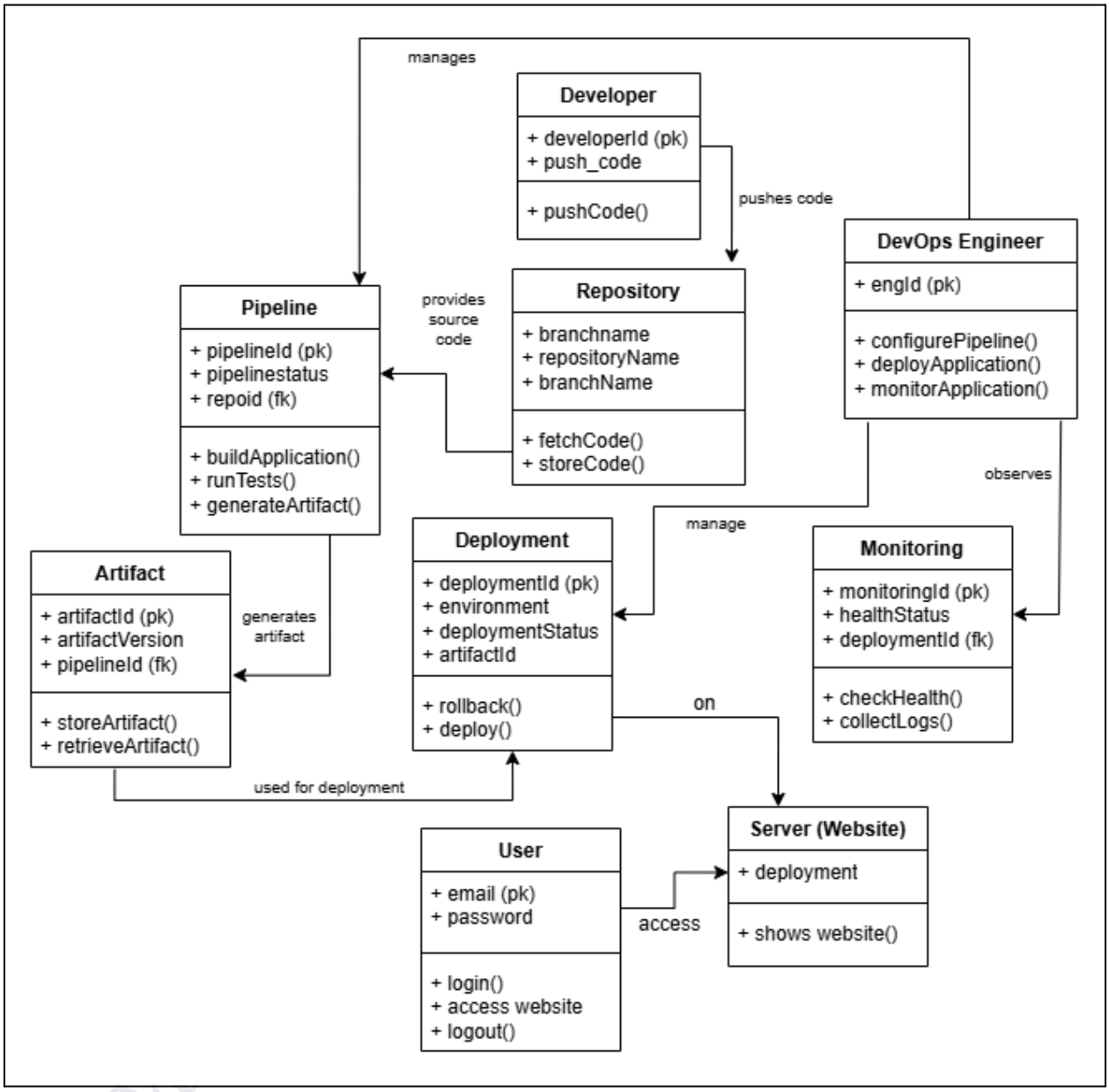



fig 8: Class diagram

## 6. Design

### a) Database

**Table 1: User**

| Sr. No. | Field Name | Data Type (Size) | Constraint | Description |
|---|---|---|---|---|
| 1 | email | email(30) | Primary Key | Email ID of user |
| 2 | password | VARCHAR (10) | Not Null | Password for authentication |

**Table 2: Developer**

| Sr. No. | Field Name | Data Type (Size) | Constraint | Description |
|---|---|---|---|---|
| 1 | developer_id | INT (11) | Primary Key | Unique identifier for developer |
| 2 | push_code | BOOLEAN | Default TRUE | Indicates ability to push code |

**Table 3: DevOps Engineer**

| Sr. No. | Field Name | Data Type (Size) | Constraint | Description |
|---|---|---|---|---|
| 1 | eng_id | INT (11) | Primary Key | Unique identifier for DevOps Engineer |

**Table 4: Repository**

| Sr. No. | Field Name | Data Type (Size) | Constraint | Description |
|---|---|---|---|---|
| 1 | repo_id | INT (11) | Primary Key | Unique identifier for repository |
| 2 | repository_name | VARCHAR (150) | Not Null | Name of repository |
| 3 | branch_name | VARCHAR (50) | Not Null | Branch name (e.g., main) |

**Table 5: Pipeline**

| Sr. No. | Field Name | Data Type (Size) | Constraint | Description |
|---|---|---|---|---|
| 1 | pipeline_id | INT (11) | Primary Key | Unique identifier for pipeline |
| 2 | pipeline_status | VARCHAR (50) | Not Null | Status (Success, Failed, Running) |
| 3 | repo_id | INT (11) | Foreign Key | References Repository(repo_id) |

**Table 6: Artifact**

| Sr. No. | Field Name | Data Type (Size) | Constraint | Description |
|---|---|---|---|---|
| 1 | artifact_id | INT (11) | Primary Key | Unique identifier for artifact |
| 2 | artifact_version | VARCHAR (50) | Not Null | Version of artifact (e.g., 1.0.1) |
| 3 | pipeline_id | INT (11) | Foreign Key | References Pipeline(pipeline_id) |

**Table 7: Deployment**

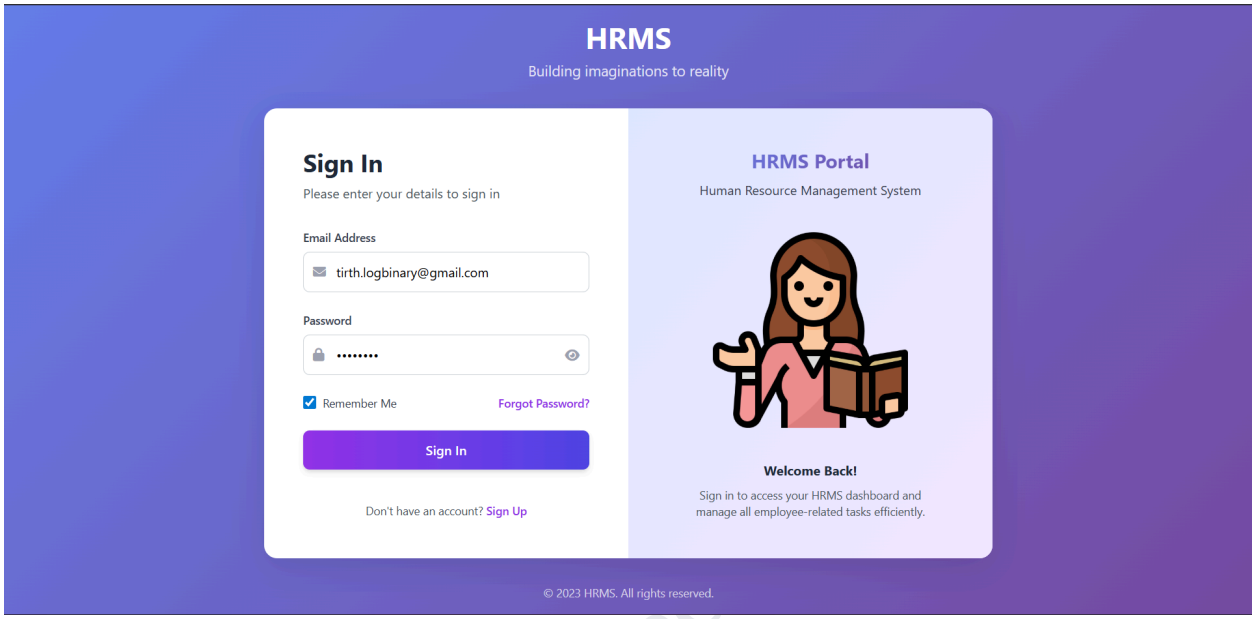
| Sr. No. | Field Name | Data Type (Size) | Constraint | Description |
|---|---|---|---|---|
| 1 | deployment_id | INT (11) | Primary Key | Unique identifier for deployment |
| 2 | environment | VARCHAR (50) | Not Null | Deployment environment (Dev, Prod) |
| 3 | deployment_status | VARCHAR (50) | Not Null | Status (Success, Failed) |
| 4 | artifact_id | INT (11) | Foreign Key | References Artifact(artifact_id) |

**Table 8: Monitoring**

| Sr. No. | Field Name | Data Type (Size) | Constraint | Description |
|---|---|---|---|---|
| 1 | monitoring_id | INT (11) | Primary Key | Unique identifier for monitoring record |
| 2 | health_status | VARCHAR (50) | Not Null | System health (Healthy, Warning, Critical) |
| 3 | deployment_id | INT (11) | Foreign Key | References Deployment(deployment_id) |

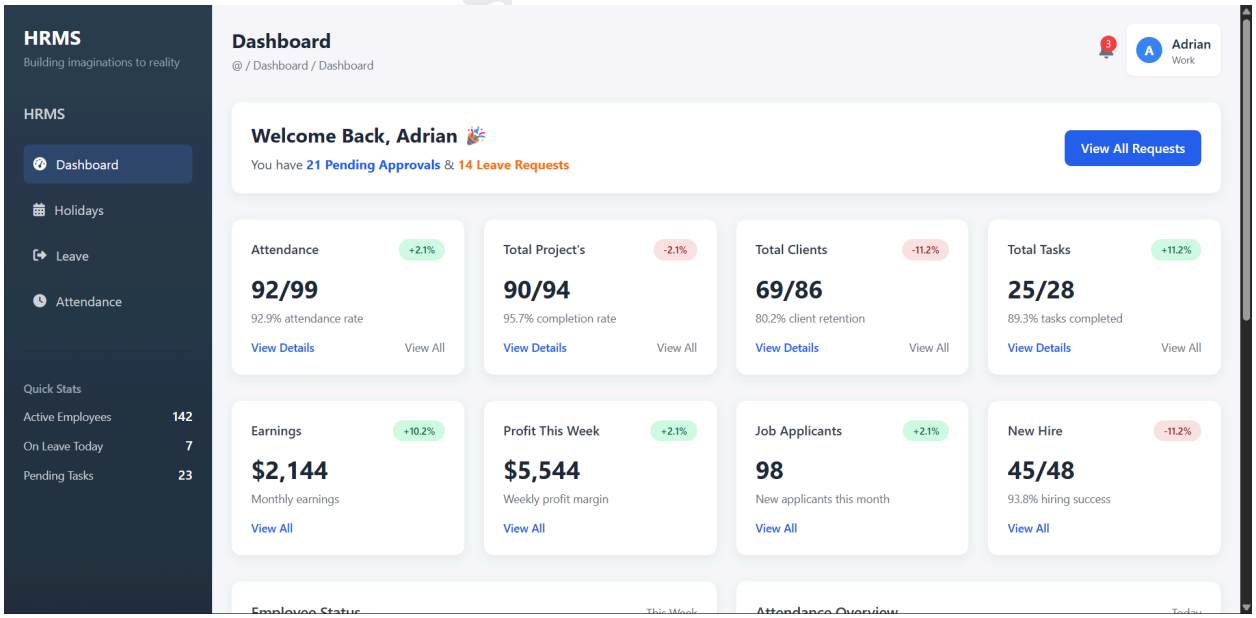
## b) Screens / Forms

### i) Login Page



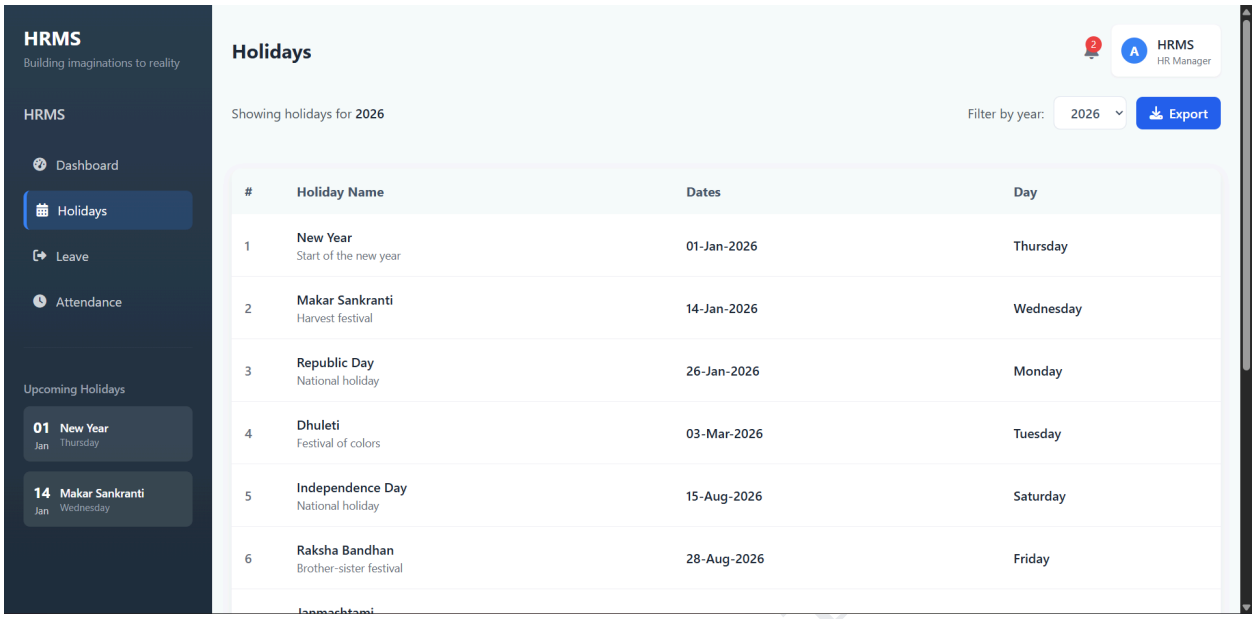

The above screen shows the HRMS login page where users enter their credentials to access the system.

### ii) Home/Dashboard Page



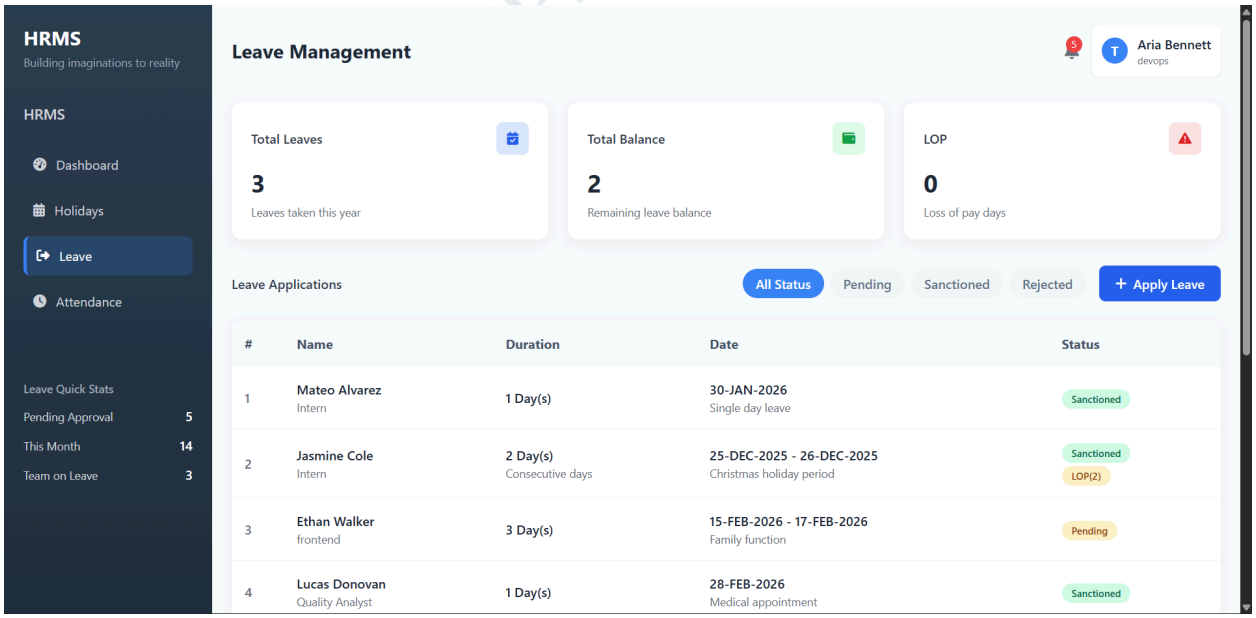

The above screen shows the HRMS dashboard displaying employee statistics, approvals, and overall system summary.

### iii) Holidays Page



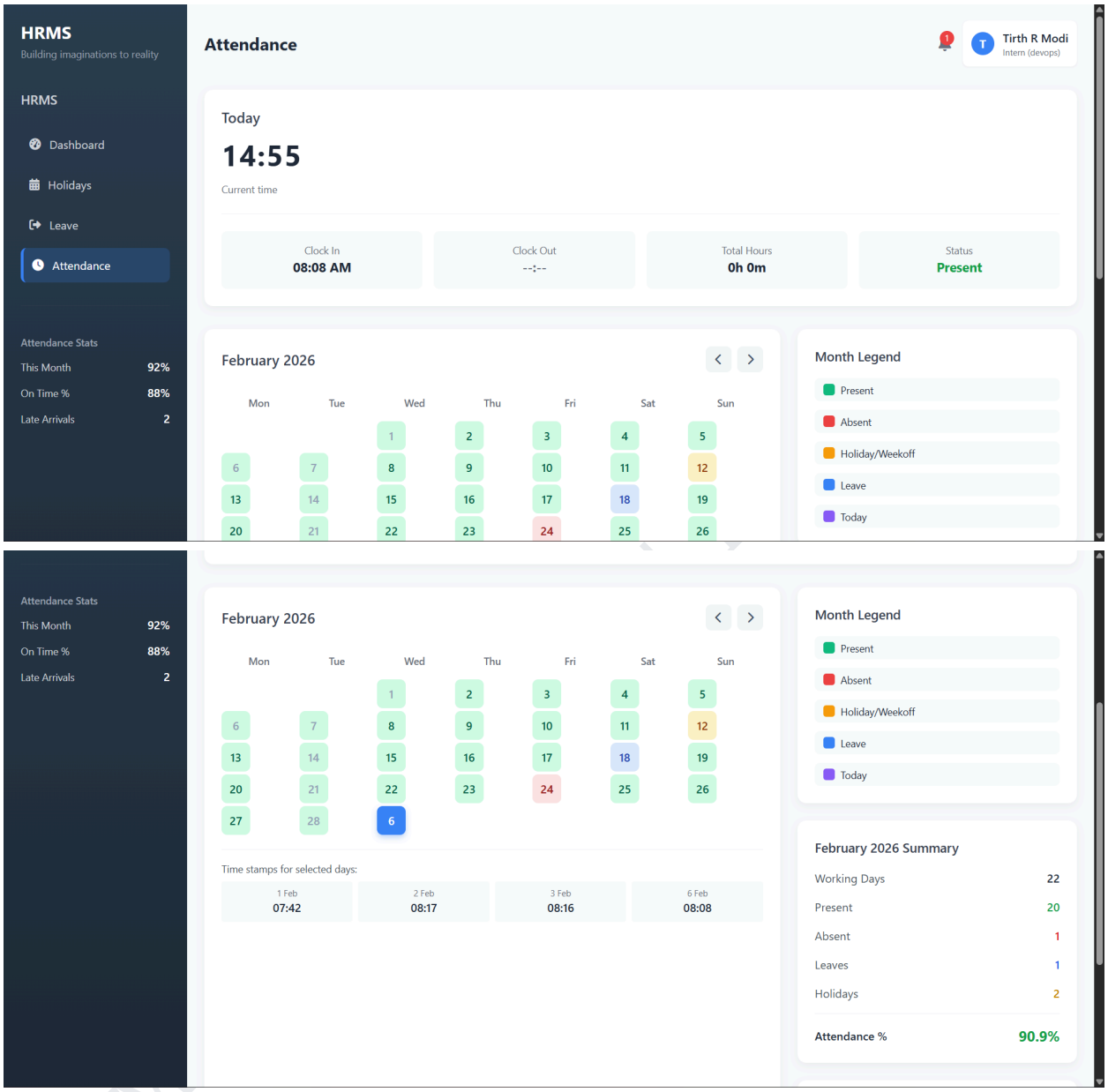

The above screen shows the holidays section where users can view company holidays with dates and details.

### iv) Leave Page



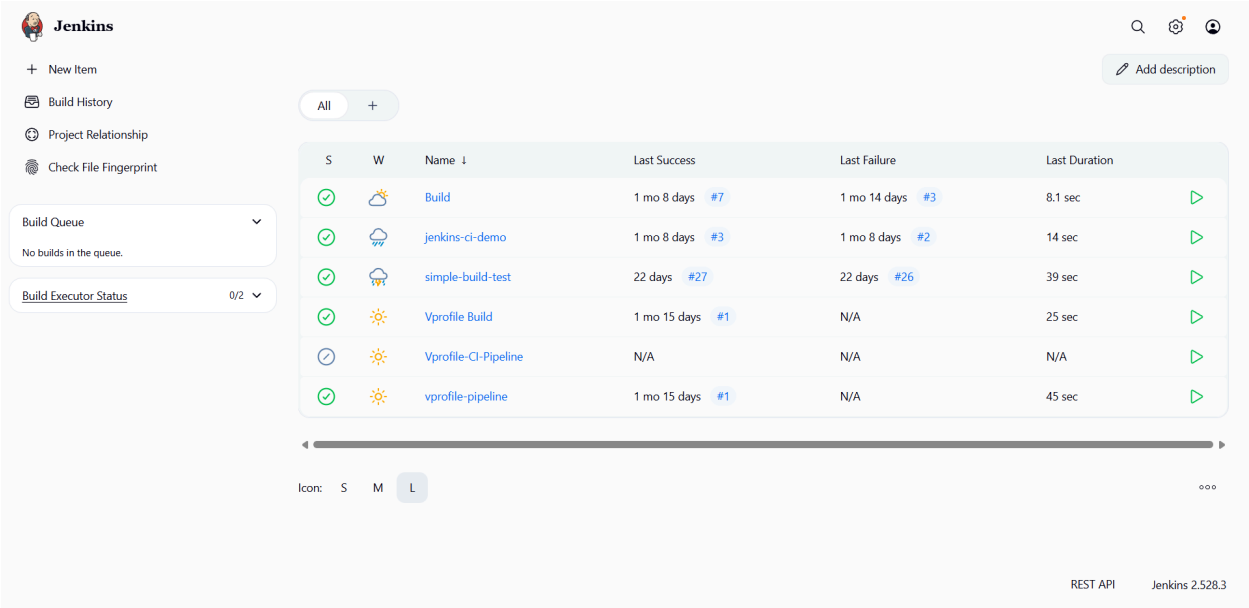

The above screen shows the leave management module where users can apply and track leave requests.

## v) Attendance Page



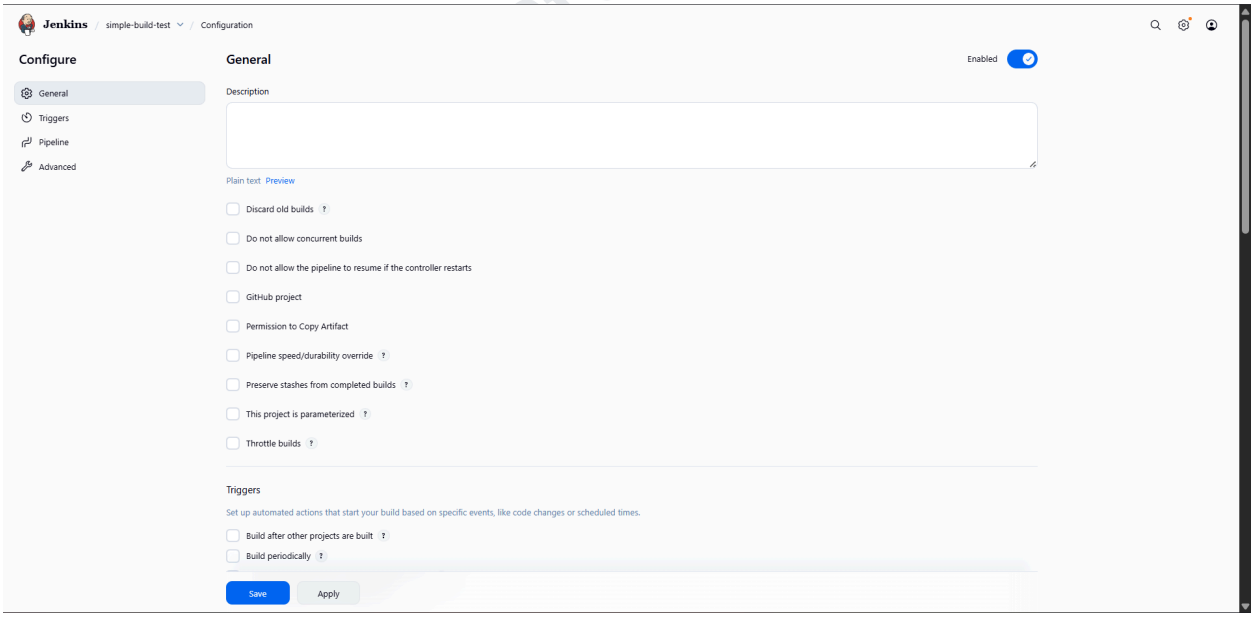

The above screens show the attendance module displaying daily time tracking and monthly attendance calendar with summary.

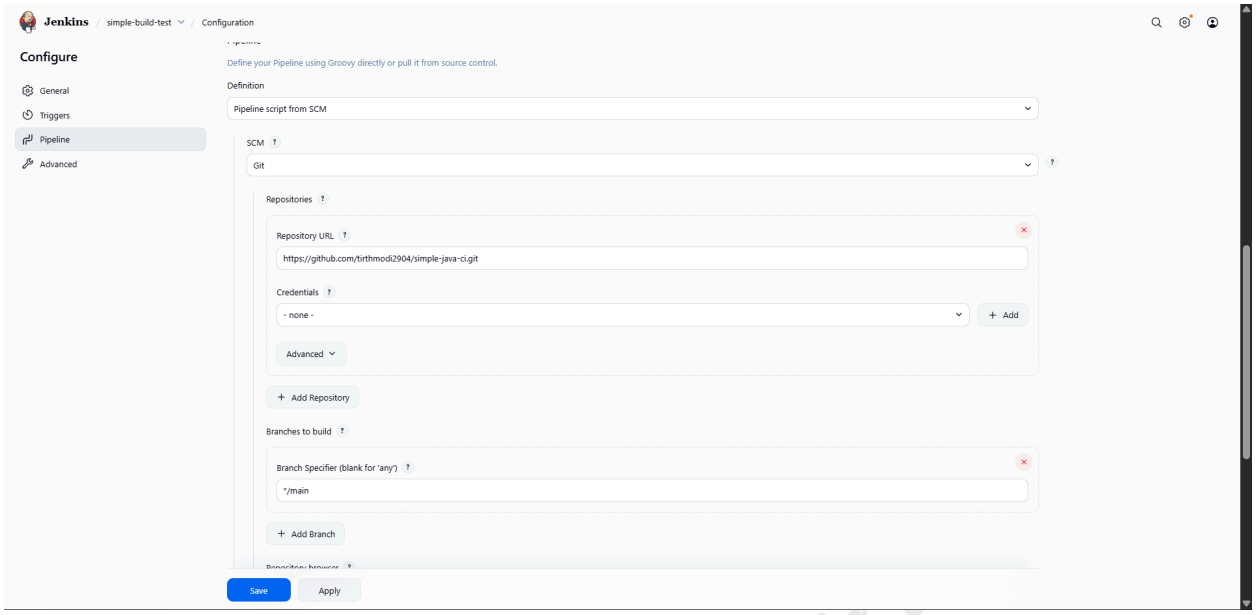
## vi) Jenkins Home Page




The above screen shows the Jenkins dashboard listing multiple jobs and their build status.

## vii) Jenkins Pipeline Creation



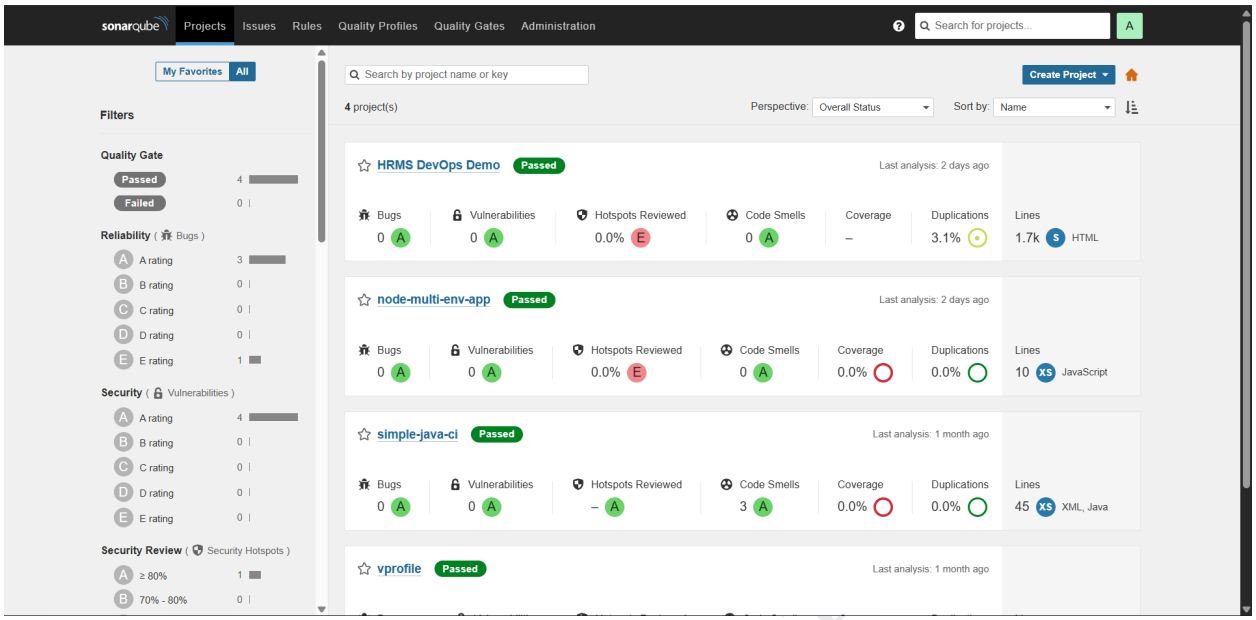



The above screens show Jenkins job configuration including general settings and pipeline setup.

### viii) Jenkins Pipeline Stages

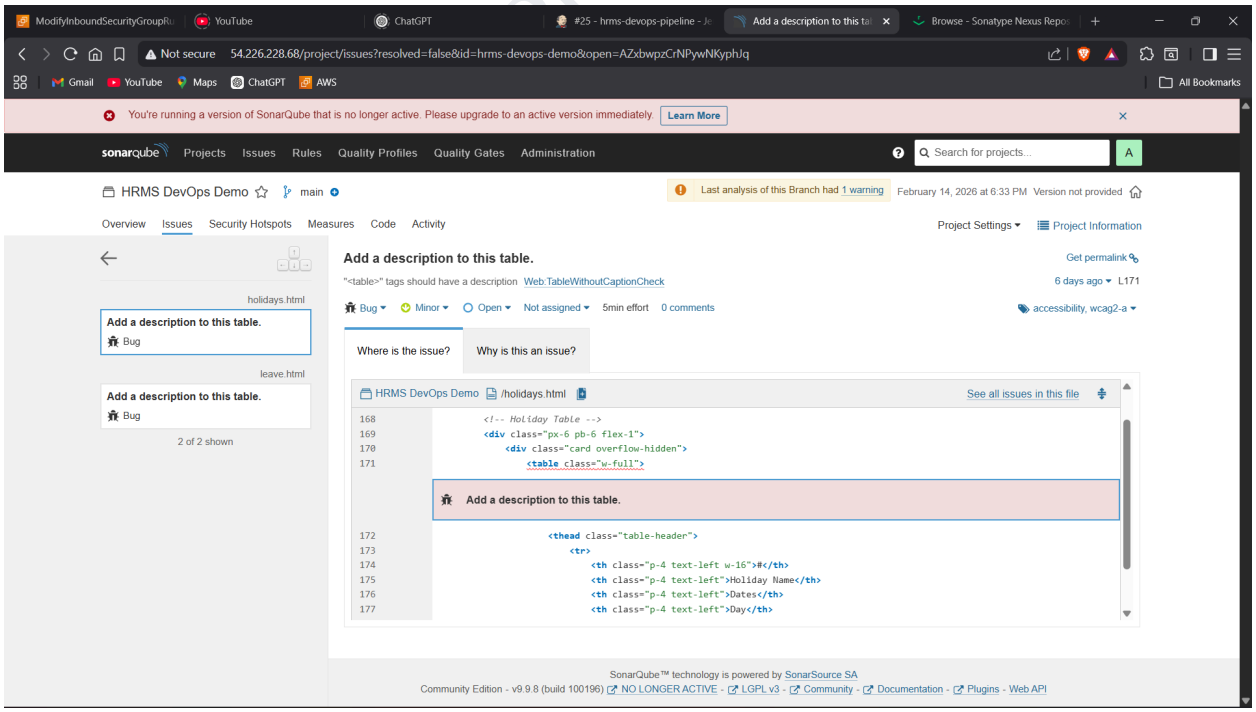

The above screen shows Jenkins pipeline execution stages such as checkout, build, test, SonarQube, and deployment status.

## ix) SonarQube



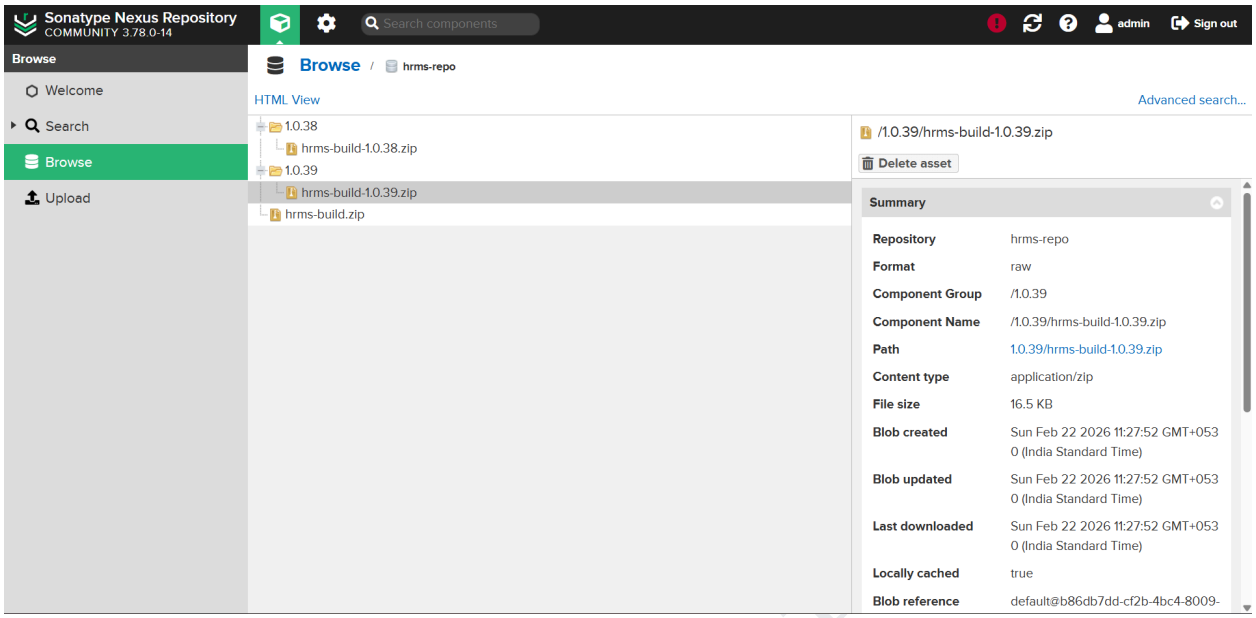

The above screen shows SonarQube dashboard displaying code quality metrics like bugs, vulnerabilities, and coverage.

## x) SonarQube Issue Page



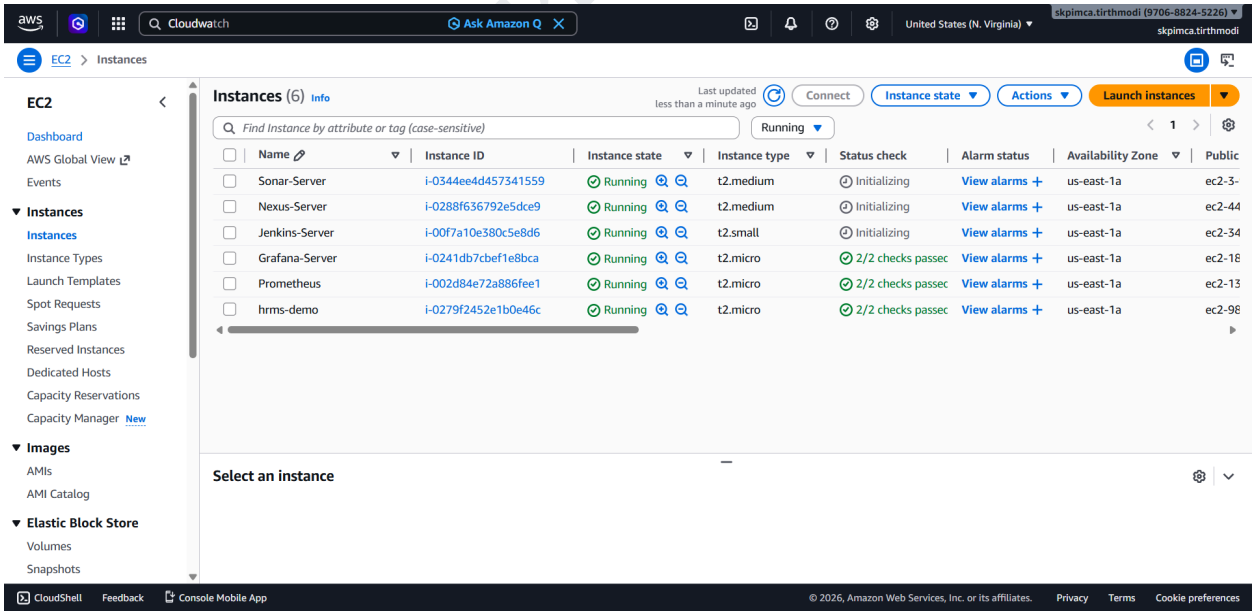

The above screen shows SonarQube code analysis highlighting issues and improvements required in the project.

## xi) Nexus Repository




The above screen shows Nexus repository storing versioned build artifacts generated by the pipeline.

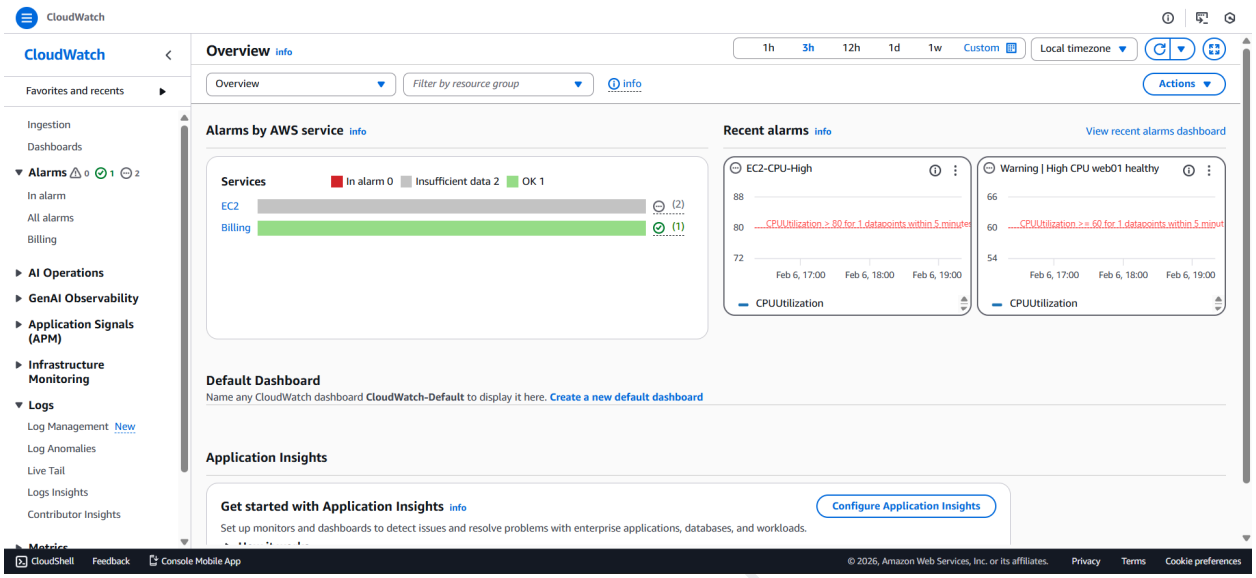
## xii) AWS EC2



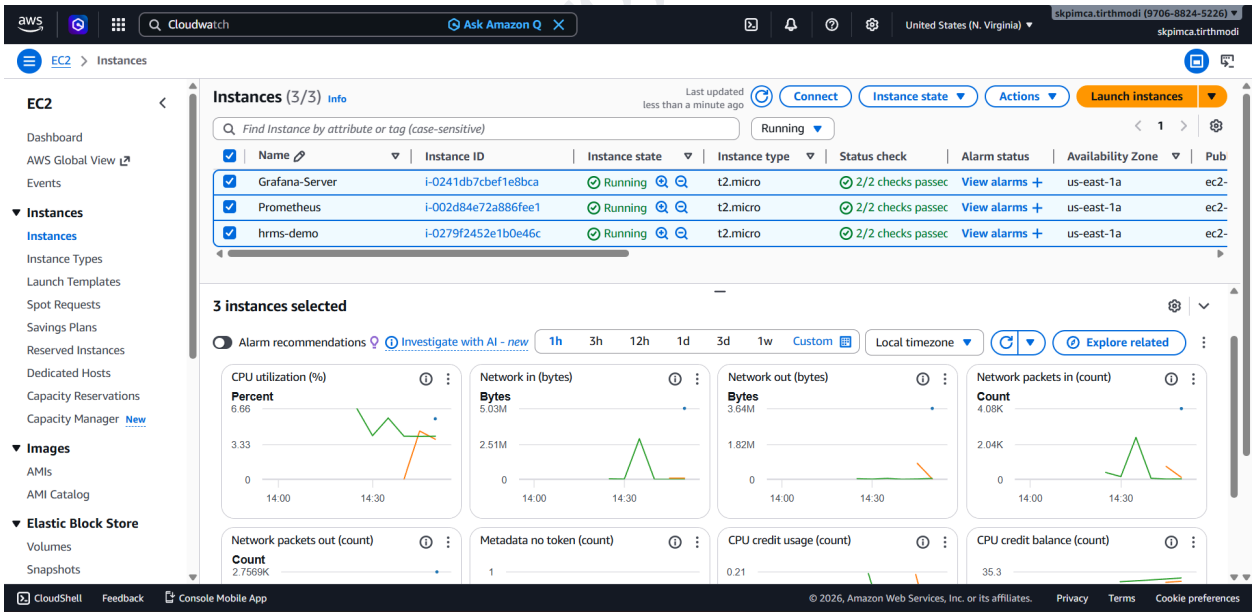

The above screen shows AWS EC2 dashboard listing multiple running instances used in the project.

## c) Reports

## i) AWS CloudWatch

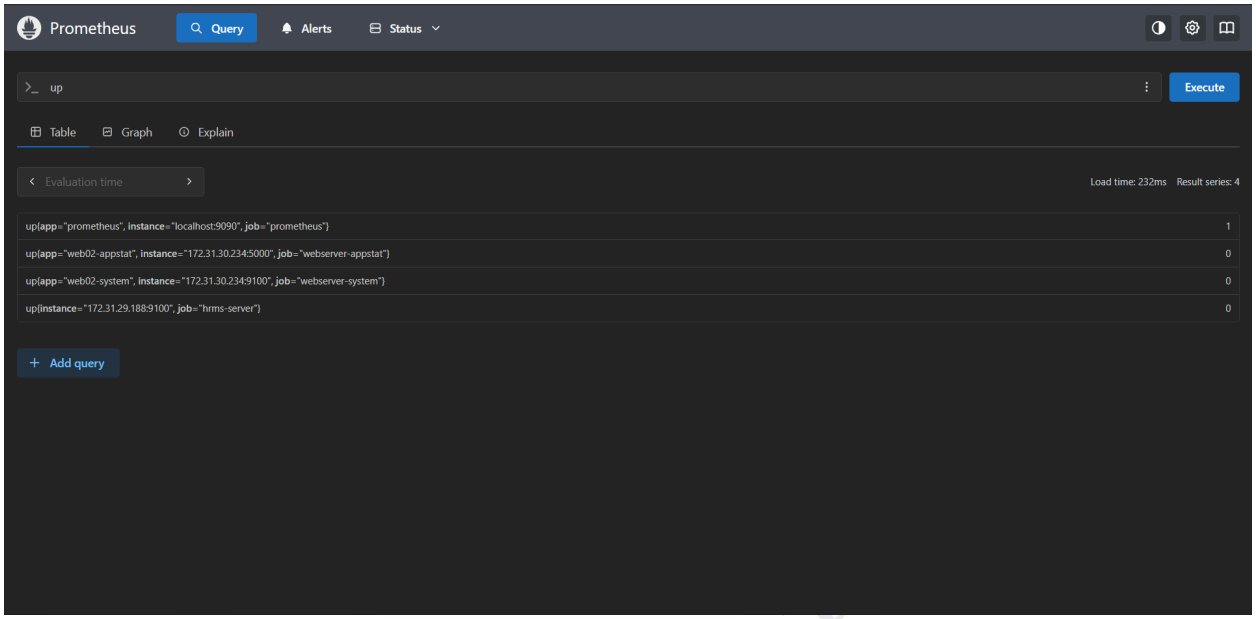



The above screen shows AWS CloudWatch monitoring service displaying alarms, logs, and system performance metrics.



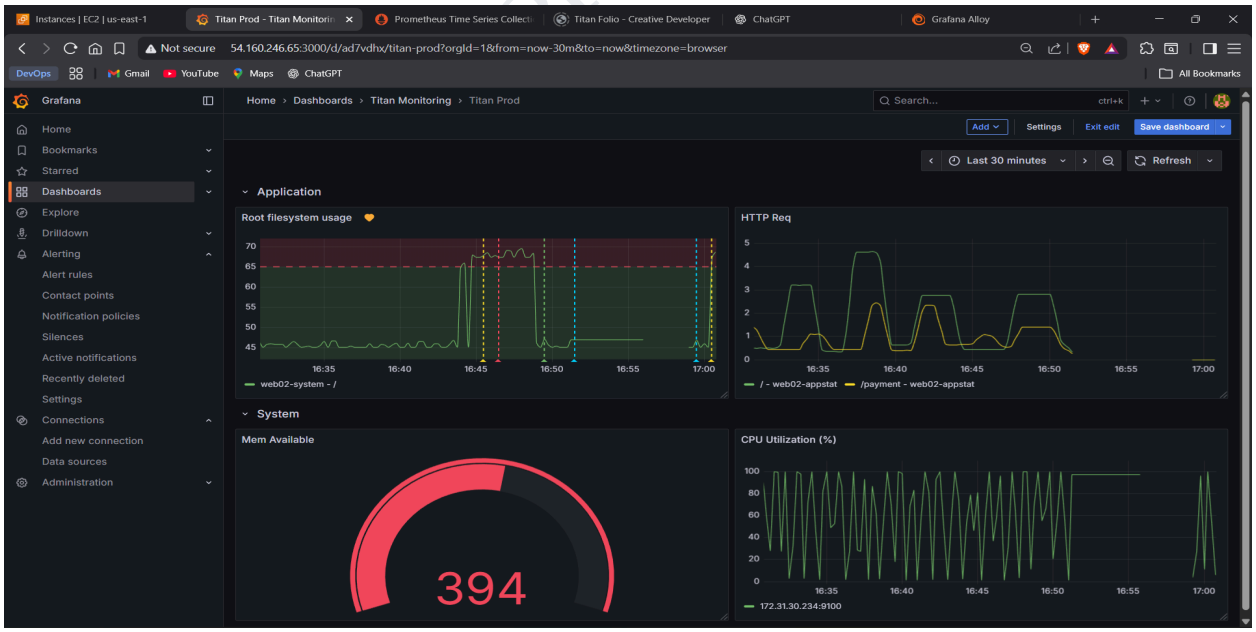

The above screen shows detailed EC2 instance monitoring including CPU, network, and resource usage graphs.

## ii) Prometheus




The above screen shows Prometheus query interface used to monitor system metrics from servers.

## iii) Grafana




The above screen shows Grafana dashboard visualizing system performance metrics like CPU, memory, and disk usage.

## 7. Testing

### a) Test Case

| Test Case ID | Description | Expected Result |
|---|---|---|
| TC01 | Push code to repository | Code should trigger automated pipeline |
| TC02 | Build application | Application should build successfully |
| TC03 | Run unit tests | Tests should execute without errors |
| TC04 | Deploy application | Application should deploy successfully |
| TC05 | Login to HRMS | User should login successfully |
| TC06 | Apply Leave | Leave request should be submitted |
| TC07 | View Attendance | Attendance data should display |
| TC08 | Monitoring logs | System logs should be visible |
| TC09 | SonarQube analysis | Code quality report should be generated successfully |
| TC10 | Quality Gate check | Pipeline should pass only if quality gate is successful |
| TC11 | Upload artifact to Nexus | Artifact should be stored successfully in Nexus repository |
| TC12 | Download artifact from Nexus | Artifact should be fetched successfully for deployment |
| TC13 | SSH connection to EC2 | Server connection should be established successfully |
| TC14 | Jenkins Pipeline | Pipeline should successful if there is no failed stage |
| TC15 | Website accessibility | HRMS website should load successfully in browser |

## 8. Future Enhancement

The proposed system successfully automates the deployment and management of the HRMS application using DevOps practices. However, the system can be further improved by introducing additional features that enhance reliability, scalability, security, and overall performance.

**Automated Rollback Mechanism**

Future versions of the system can include an automated rollback feature that restores the previous stable version of the application if a deployment fails. This will reduce downtime, improve reliability, and ensure quick recovery from deployment issues.

**Multi-Environment Deployment Support**

The system can be extended to support multiple environments such as development, testing, staging, and production. This will enable better testing, validation, and quality assurance before releasing updates to end users.

**Enhanced Security Features**

Security can be strengthened by implementing role-based access control, secure authentication methods, encrypted data transmission, and automated vulnerability scanning. These measures will help protect sensitive HR data and improve system trustworthiness.

**Multi-Cloud Deployment Integration**

The system can be enhanced to support deployment across multiple cloud platforms. Multi-cloud integration will improve scalability, increase flexibility, and provide better disaster recovery and availability.

**Container-Based Deployment Integration**

Future deployment processes can incorporate container-based technologies, allowing applications to run consistently across different environments. This will simplify deployment, improve portability, and support modern DevOps practices.

**Performance Optimization Automation**

The system can include automated performance testing and optimization during the deployment pipeline. This will help identify performance issues early and ensure efficient application operation under different workloads.

## 9. References/Bibliography


Process Model: https://dev.to/nolunchbreaks_22/the-rising-tide-of-platform-engineering-reshaping-security-in-modern-devops-3ep5

Jenkins: https://www.jenkins.io/

SonarQube: https://www.geeksforgeeks.org/devops/sonarqube/

Nexus Repository: https://help.sonatype.com/en/sonatype-nexus-repository.html

Prometheus: https://prometheus.io/docs/introduction/overview/

Grafana: https://grafana.com/docs/

AWS EC2: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html

AWS Cloudwatch: https://aws.amazon.com/cloudwatch/

CI/CD Workflow: https://ghazanfaralidevops.medium.com/jenkins-continuous-integration-pipeline-using-aws-ec2-instances-a50af8a26698